

FeatureLeakageLens

Product Requirements Document — v0.2.0

Pre-training feature leakage auditor for tabular ML datasets

Field	Value
Status	Released — v0.2.0
Author	Sidharth Kriplani
Language	Python 3.10 / 3.11 / 3.12
License	MIT
Dependencies	pandas, numpy
Tests	13 unit tests, 4 test classes
PyPI	featureleakagelens

1. Problem Statement

Feature leakage is the most reliably invisible failure mode in supervised ML. A model trained on a post-outcome column produces metrics that look like a breakthrough — AUC above 0.98, near-perfect precision — because it is essentially predicting the label from a compressed copy of the label. Nothing in the training loop signals this. The model is not miscalibrated; it is correct on training data in a way that cannot generalise.

The failure surfaces in production. The model makes predictions using a feature value that doesn't exist yet at prediction time. The feature pipeline returns null or stale data. Predictions degrade to chance level. By that point, months of development, stakeholder communication, and business decisions built on the model's offline metrics are already sunk.

The review step that should catch this is informal. There is no standard checklist, no structured output to attach to a model card, and no CI gate. A data scientist reviews the columns they think of reviewing. The ones they don't think of become production failures.

2. Goals and Non-Goals

Goals:

- Single function call — `audit_dataframe(df, config)` — that checks six leakage patterns before training
- Tiered checks: cheap name/structure checks first, statistical checks second, temporal last
- FAIL for structural violations (temporal); WARN for statistical suspicions requiring domain confirmation
- Evidence dicts on every finding — quantitative detail, not just status
- Three output formats: JSON (CI gates), Markdown (model card attachment), HTML (human review)

Non-Goals:

- Pipeline leakage (scaler/encoder fit on full dataset) — requires code inspection, not DataFrame audit
- Group leakage in k-fold CV — requires fold assignments not present in a single DataFrame
- Target encoding leakage — requires lineage tracing of how features were computed
- Proving leakage — every WARN requires domain confirmation; the tool flags, humans decide

3. Target Users

User	Use case	How they use output
ML Engineer / Data Scientist	Run before any <code>model.fit()</code> ; catch leakage before training	Interpret offline metrics as JSON to model card or PR

ML Reviewer / Team Lead	Require audit as part of model sign-off	Gate sign-off on report.status != FAIL
MLOps / ML Platform	CI gate: fail training pipeline on FAIL status	Parse report.status from JSON; exit non-zero on FAIL
Interview candidate	Demonstrate systematic ML engineering process	Present PDF and discuss design decisions

4. Check Specifications

Tier	Check	Config fields	Default	FAIL condition	WARN condition
1	Post-outcome name heuristic	target_col	—	—	Col name matches keyword list
1	ID / proxy scan	id_threshold	0.95	—	n_unique/n_rows ≥ threshold
2	Target correlation	high_corr_threshold	0.95	—	Pearson r ≥ threshold
2	Categorical proxy	cat_proxy_threshold	0.50	—	max_rate – min_rate ≥ threshold
2	Split distribution	split_col, distribution_shift_threshold	0.15	—	Shift ≥ threshold; INSUFFICIENT_INF
3	Temporal availability	outcome_time_col, feature_time_cols	—	feature_ts > outcome_ts on any row	

5. Input / Output Model

Input:

Field	Type	Required	Description
target_col	str	Yes	Name of the binary target column
split_col	str None	No	Column with train/test labels (enables split distribution scan)
outcome_time_col	str None	No	Outcome event timestamp column (required for temporal check)
feature_time_cols	dict[str,str]	No	Maps feature column → its timestamp column
high_corr_threshold	float	No	Pearson r threshold for target correlation scan (default 0.95)
cat_proxy_threshold	float	No	Target rate gap threshold for categorical proxy scan (default 0.50)
id_threshold	float	No	n_unique/n_rows threshold for ID/proxy scan (default 0.95)
distribution_shift_threshold	float	No	Shift threshold for split distribution scan (default 0.10)

Output — LeakageReport:

Field	Type	Description
status	str	Worst finding: FAIL › WARN › INSUFFICIENT_INPUT › PASS
findings	list[LeakageFinding]	Per check per column: check_name, status, severity, evidence, column_name
summary	dict	rows, columns, finding_count, warn_count, fail_count
to_json() / to_markdown() / to_html()	str	Three output formats
save(path, stem)	None	Writes .json, .md, .html to path

6. Acceptance Criteria

Criterion	Quantitative bar	Verified
Temporal check fires on feature_ts > outcome_ts	status FAIL, evidence future_rows > 0	Yes
Temporal check is PASS when all feature_ts <= outcome_ts	status PASS on temporal finding	Yes
Name heuristic fires on 'target_proxy_paid' column	check_name post_outcome_name_heuristic	Yes
High correlation (r ≈ 0.99) triggers correlation WARN	test_high_correlation_warns	Yes
Low correlation (r ≈ 0.0) produces no correlation finding	test_low_correlation_does_not_warn	Yes

Large distribution shift (mean diff = 10 SDs) triggers <code>WARN</code>	<code>test_large_distribution_shift_warns</code> passes	Yes
No <code>split_col</code> → <code>INSUFFICIENT_INPUT</code> on split distribution check	<code>test_no_split_col_gives_insufficient_input</code> passes	Yes
<code>save()</code> writes exactly 3 files (.json, .md, .html)	<code>test_save_writes_three_files</code> passes	Yes
Missing <code>target_col</code> → FAIL on input schema check	<code>test_missing_target_fails</code> passes	Yes
13/13 unit tests pass on Python 3.10, 3.11, 3.12	CI matrix green	Yes

7. Failure Scenarios That Drove Design Decisions

Scenario	Historical failure mode	How FeatureLeakageLens addresses it
payment_received_flag in training data after default decision; model prediction heuristics from WARN overposed to produce FAIL if timestamp provided	Flagging data	Has heuristic from WARN overposed to produce FAIL if timestamp provided
segment column created from risk segment assigned using default rate; categorical as proxy	Risk segment	Categorical as proxy can WARN (target rate gap near 1.0)
customer_id not dropped from features	Time features	Time features memorises training IDs; perfect ID/proxy as proxy (AUC ~ 1.0)
Train/test split correlated with time	Retention	Retention customers in test, old in train; feature distribution shift WARN on numeric features
default_probability_v1 from prior model score included as feature; end correlation score	Prior model	Prior model score included as feature; end correlation score WARN (r very high with target)

8. Summary

Dimension	v0.2.0 Status
All 6 checks across 3 tiers	Done
13 unit tests passing	Done
CI matrix Python 3.10/3.11/3.12	Done
JSON + MD + HTML output	Done
Evidence dicts on all findings	Done
Configurable thresholds in LeakageAuditConfig	Done
Temporal FAIL / all others WARN distinction	Done
PyPI ODC publish workflow	Done

Resume-safe claim: Built FeatureLeakageLens, a pre-training feature leakage auditor for tabular ML datasets that checks for post-outcome name heuristics, target correlation, categorical target-rate proxies, future timestamp leakage, ID/proxy columns, and train/test distribution shift, producing structured JSON/Markdown/HTML audit reports with per-finding PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary.